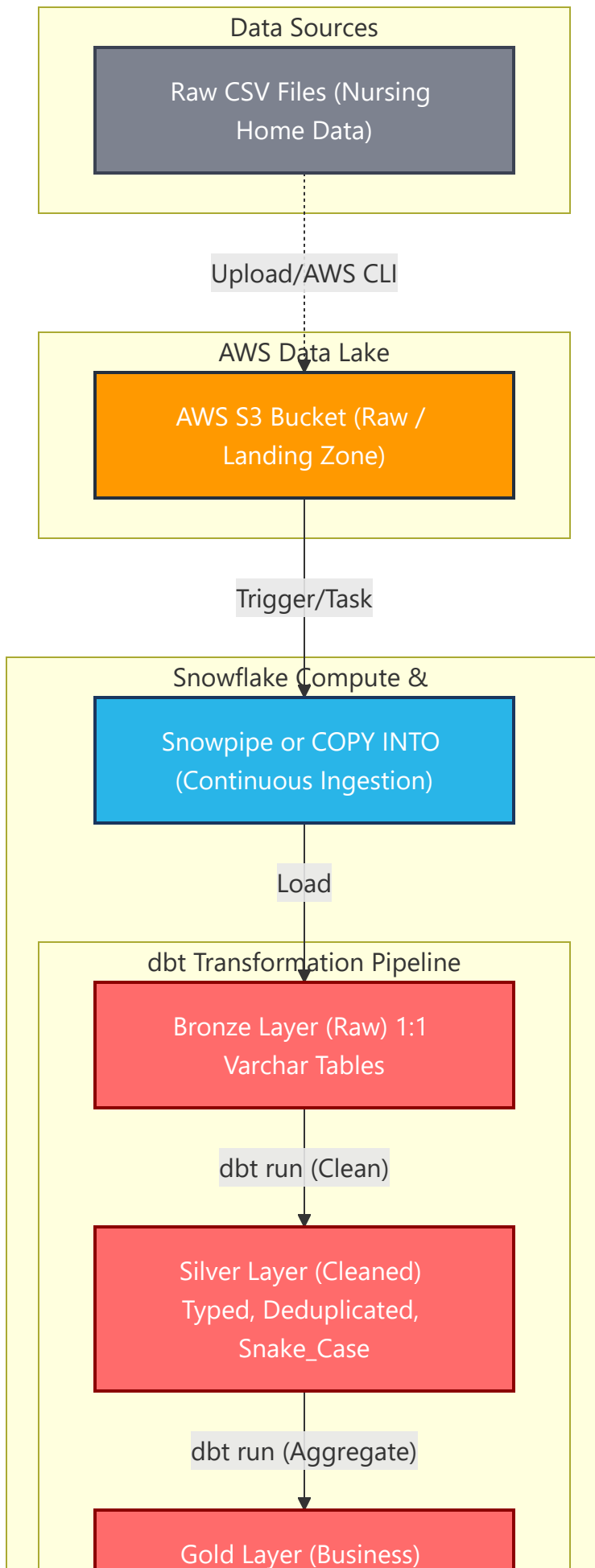
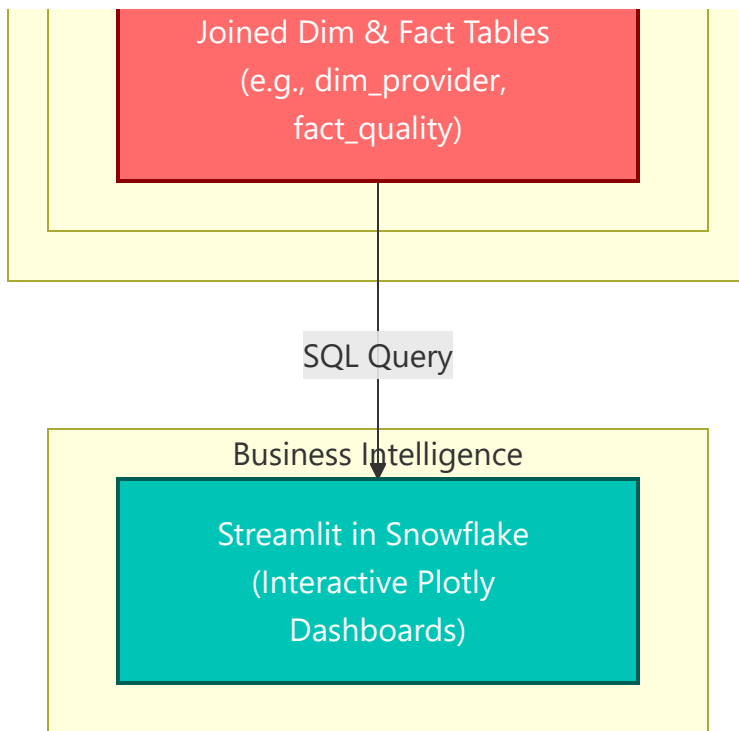


Idea 1: AWS, Snowflake, and DBT





How the Pipeline Works:

1. Ingestion & Storage (AWS S3)

- **The Landing Zone:** Raw CSV files are dropped into an S3 bucket (`s3://nursing-home-data/landing/`). Because S3 is incredibly cheap, this acts as an immutable, permanent data lake and historical archive.
- **Cross-Cloud Integration:** Snowflake securely integrates with S3 bucket using an **External Stage** and a Storage Integration object.

2. The Snowflake Compute Engine

- **Snowpipe:** Configure an S3 Event Notification to trigger a Snowpipe automatically the second a new CSV is dropped into the bucket. Snowpipe quickly pulls that CSV directly into Snowflake.

3. Data Transformation (dbt Medallion Architecture)

Instead of writing sprawling ad-hoc SQL scripts, `dbt` acts as the orchestrator to transform the data systematically inside Snowflake:

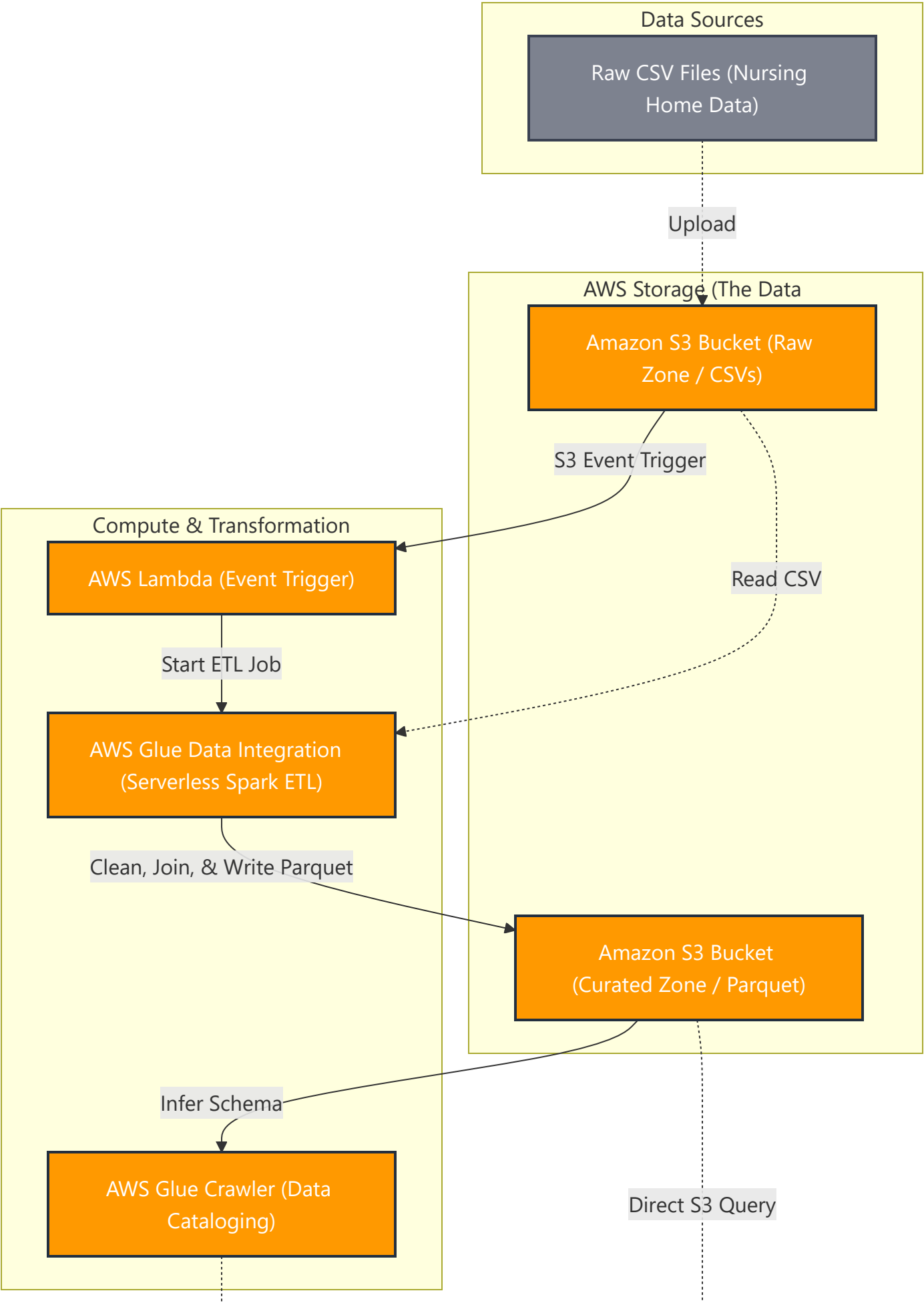
- **Bronze Layer (Raw):** Data lands exactly as it appears in the CSV. Every column is natively kept as `VARCHAR/STRING` . (`NH_ProviderInfo_Raw` , `NH_QualityMsr_MDS_Raw`)
- **Silver Layer (Cleaned):** `dbt` standardizes the mess. It casts `Reported Nurse Staffing Hours` to a `FLOAT` , strips bad characters out of column names (e.g., converting spacing to snake_case like `number_of_certified_beds`), handles `NULL` values, and deduplicates the data if a CSV was uploaded twice.

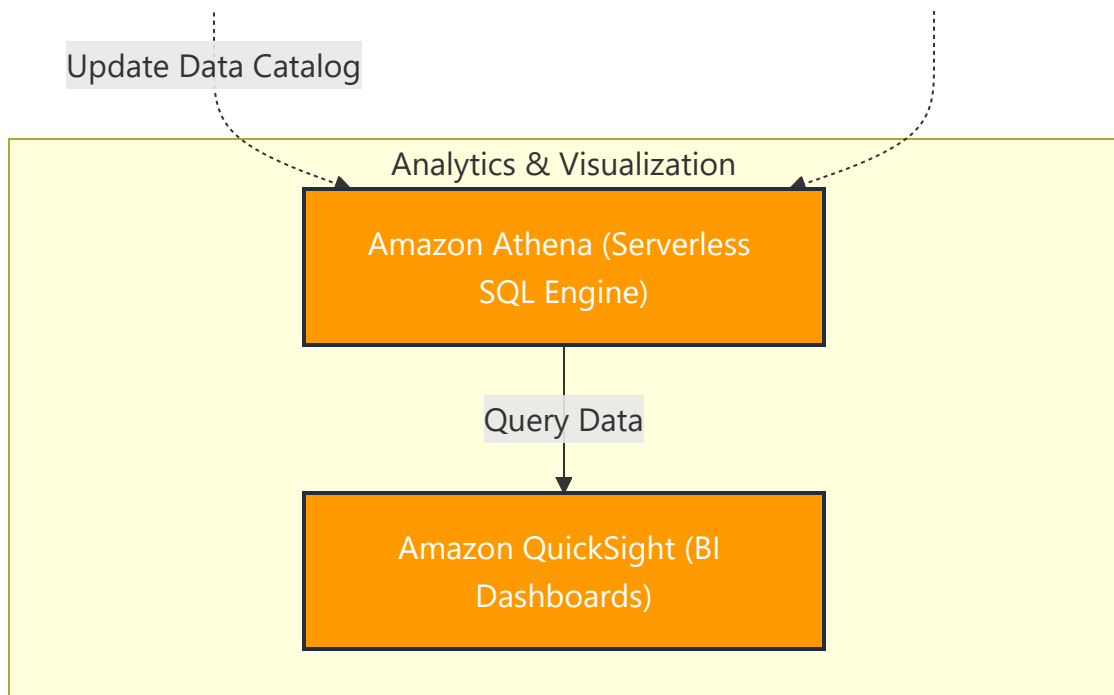
- **Gold Layer (Business Prepared):** This is where constraints and joins happen. `dbt` takes the pristine Silver data and builds dimensional data models. It generates the exact `NH_ProviderInfo_Oct2024` and `NH_QualityMsr_MDS_Oct2024` tables, joined perfectly on the `CCN` facility ID, making them incredibly fast for downstream tools to query.

4. Business Intelligence Visualization

- **Streamlit:** With the tables cleanly modeled in the Gold layer, a Streamlit app can query Snowflake directly to generate interactive views (e.g., a map chart showing average staffing levels by state, or a line chart showing quality measures over the four historical quarters).

Idea 2: Full AWS





Why this stack and how it works:

1. Zero Infrastructure Management

Instead of provisioning a dedicated data warehouse, we rely entirely on **Amazon S3** as the permanent storage layer. It acts as both the raw landing zone and the finalized database storage.

2. Event-Driven Transformation (AWS Lambda & Glue)

- **The Trigger:** The moment a new bulk dataset of CSVs drops into the **Raw Zone S3** bucket, it automatically triggers an **AWS Lambda** function.
- **The Processor:** Lambda instantly kicks off an **AWS Glue** job. Glue is a fully managed, serverless Spark environment. It reads the raw CSVs, cleans the column headers, casts data types, joins the **ProviderInfo** and **MDS** tables via the **CCN** facility ID, and aggregates the averages (e.g., Staffing by State).
- **The Output:** Crucially, Glue writes the final, clean tables back into the **Curated Zone S3** bucket as **Apache Parquet files**. Parquet is a heavily compressed, columnar format that is drastically faster and cheaper to query than CSV.

3. Data Cataloging (AWS Glue Crawler)

Once the Parquet files are created, an **AWS Glue Crawler** automatically scans the new files, identifies the schema (columns, data types), and registers them as virtual SQL tables in the AWS Data Catalog.

4. Serverless Analytics & BI (Athena & QuickSight)

- **Amazon Athena:** No database needed to write SQL. Athena allows me to query the Parquet files sitting in my S3 bucket directly using standard SQL. And only pay a few cents per Terabyte scanned!
- **Amazon QuickSight:** AWS's native visualization tool. It connects directly to Athena. You can drag and drop to build maps, bar charts, and BI dashboards showing the relationships between nurse staffing levels and facility types. QuickSight also features "SPICE" (an in-memory caching engine) to make the dashboard dashboards load instantly for users, without having to re-query the S3 bucket every time.

The Verdict: Snowflake + dbt idea is highly structured, more modern, and excellent for collaboration (version controlled as well). AWS Native idea is infinitely scalable, near-zero cost when idle, and requires absolutely no database administration